

Tema 01 · JDBC: Statement, PreparedStatement y CallableStatement

Clases 014 (24-06) y 015 (29-07) · Teoría: «Base de datos – Parte I, II y III»

Programación Avanzada · Prof. Macrino Matías · Universidad del Salvador – Sede Pilar · 2.º cuatrimestre

Ubicación del tema: estas dos clases cierran el primer tramo de la materia, pero JDBC es la base de todo lo que sigue y el parcial del 2.º cuatrimestre pide explícitamente «los tres métodos de JDBC (Statement, PreparedStatement y CallableStatement)». Por eso este tema es el primero de la carpeta.

1. Qué es JDBC

JDBC (Java Database Connectivity) es la API estándar de Java para hablar con bases de datos relacionales. Es un conjunto de interfaces del paquete `java.sql` (`Connection`, `Statement`, `ResultSet`...). Cada fabricante de base de datos (MySQL, SQL Server, PostgreSQL...) entrega un **driver** que implementa esas interfaces. Gracias a eso el código Java es casi el mismo para cualquier motor: lo único que cambia es el driver y la URL.

Existen cuatro tipos de drivers. El **tipo 4** (el que usamos) es 100% Java y se comunica directamente con el protocolo de red de la base de datos: no necesita software nativo instalado y es portable. Es el estándar en arquitecturas de tres capas.

2. Los pasos de cualquier acceso a datos con JDBC

Paso	Qué se hace	Clase o método
1	Cargar el driver (opcional en drivers modernos)	<code>Class.forName("com.mysql.cj.jdbc.Driver")</code>
2	Abrir la conexión	<code>DriverManager.getConnection(url, usuario, clave) → Connection</code>
3	Crear la sentencia	<code>createStatement()</code> / <code>prepareStatement(sql)</code> / <code>prepareCall("{call sp(?)})"</code>
4	Ejecutar	<code>executeQuery()</code> (SELECT → <code>ResultSet</code>), <code>executeUpdate()</code> (INSERT/UPDATE/DELETE → filas afectadas), <code>execute()</code>
5	Recorrer el resultado	<code>while (rs.next()) { rs.getString("col") ... }</code>
6	Cerrar los recursos	<code>rs.close()</code> , <code>st.close()</code> , <code>con.close()</code> — siempre en este orden: primero lo más interno

La **URL JDBC** de MySQL tiene la forma `jdbc:mysql://host:puerto/nombre_base`. Una **SQLException** puede ocurrir en casi todos los pasos y es una excepción controlada (checked): hay que capturarla o declararla con `throws`.

3. Los tres tipos de sentencia

	Statement	PreparedStatement	CallableStatement
Para qué sirve	SQL fijo, sin parámetros	SQL con parámetros ? , precompilado	Llamar a un procedimiento almacenado (SP)

	Statement	PreparedStatement	CallableStatement
Cómo se crea	con.createStatement()	con.prepareStatement(sql)	con.prepareCall("{call sp(?,?)}")
Cómo se ejecuta	st.executeQuery(sql) — el SQL se pasa al ejecutar	ps.executeQuery() — sin argumentos: el SQL ya se pasó al crearlo	cs.execute()
Parámetros	No: se concatena el texto (peligroso)	ps.setString(1, ...), ps.setInt(2, ...)	IN con setXxx, OUT con registerOutParameter + getXxx
Seguridad	Vulnerable a inyección SQL	Protege contra inyección SQL	Protege (parámetros) y centraliza lógica en la base
Rendimiento	Se compila en cada ejecución	Se compila una vez y se reutiliza	Altamente performante: el SP ya está en el servidor
Cuándo usarlo	Consultas fijas (ej. listar todo)	Todo lo que recibe datos del usuario	Lógica de negocio dentro de la base

Métodos de ejecución

- **executeQuery(sql)** → devuelve un ResultSet. Se usa para SELECT.
- **executeUpdate(sql)** → devuelve un int con la cantidad de filas afectadas. Se usa para INSERT, UPDATE y DELETE.
- **execute(sql)** → devuelve true si la sentencia generó un ResultSet (SELECT) y false si no (INSERT/UPDATE/DELETE). Es el método genérico; con CallableStatement es el que se usa.

4. La inyección SQL (SQL Injection)

Si el SQL se arma **concatenando** lo que escribió el usuario, el usuario puede meter código SQL. Ejemplo (el que muestra la clase en el método sql): el login arma `SELECT * FROM usuarios WHERE username = '+user+' AND password = '+pass+'`. Si en la contraseña se escribe `' OR '1' = '1'` la consulta queda ... `password = " OR '1' = '1'`, que es siempre verdadera: el atacante entra sin saber la clave. Con **PreparedStatement** los valores viajan separados del SQL y siempre se tratan como datos, nunca como código, por eso es inmune.

5. El ResultSet

- Es el resultado de un SELECT. Por defecto es **solo avance y solo lectura** (TYPE_FORWARD_ONLY + CONCUR_READ_ONLY).
- El cursor arranca **antes de la primera fila**; next() lo mueve y devuelve true mientras haya una fila.
- Se leen las columnas con getXxx("nombre") o getXxx(posicion). **Las posiciones empiezan en 1**, no en 0.
- Métodos de posición: isFirst(), isBeforeFirst(), isLast(), isAfterLast(), getRow().
- **ResultSetMetaData** (rs.getMetaData()) informa sobre las columnas: getColumnCount(), getColumnIndex(i), getColumnTypeName(i). Permite armar listados genéricos.
- Con createStatement(tipo, concurrencia) se cambia el comportamiento: TYPE_SCROLL_INSENSITIVE / TYPE_SCROLL_SENSITIVE (moverse en ambos sentidos) y CONCUR_UPDATABLE (modificar filas desde el ResultSet).
- Si el mismo Statement ejecuta otra consulta, el ResultSet anterior se cierra solo. Para dos ResultSet a la vez se necesitan dos Statement.

6. Procedimientos almacenados y CallableStatement

Un **procedimiento almacenado (stored procedure, SP)** es un bloque de código SQL guardado dentro de la base de datos, con nombre y parámetros. Ventajas: rendimiento (ya está compilado en el servidor), seguridad y reutilización de la lógica. Sus parámetros pueden ser **IN** (entrada), **OUT** (salida) o

INOUT.

Desde Java se invoca con **CallableStatement**: con.prepareCall("{ call nombre_sp(?, ?, ?) }"). Los parámetros de entrada se cargan con setXxx(índice, valor); los de salida se **registran antes de ejecutar** con registerOutParameter(índice, Types.XXX); se ejecuta con execute() y los valores de salida se leen con getXxx(índice).

7. Cierre de recursos y transacciones (adelanto)

Statement y ResultSet los cierra el recolector de basura, pero **se deben cerrar explícitamente** con close() para liberar recursos. Orden: primero ResultSet y Statement, **luego** la Connection (cerrar la conexión primero provoca errores). Hoy se recomienda **try-with-resources**, que cierra solo y en el orden correcto. Las transacciones (setAutoCommit(false), commit(), rollback()) se desarrollan en el Tema 02.

Código de las clases, explicado línea por línea

Cada recuadro gris reproduce líneas reales del archivo (con su número de línea) y el recuadro verde de abajo explica qué hacen y por qué están escritas así.

Clase 014 - 24-06: Statement vs. PreparedStatement

Se crea una base **universidad** con la tabla **estudiantes** y se programan: un alta y una búsqueda con **Statement** (concatenando texto), una demostración de inyección SQL y una actualización con **PreparedStatement**. La clase Pruebas es el main que las ejecuta.

014 - Clase 24-06 / pom.xml

Qué es y para qué sirve: Archivo de configuración de Maven (POM) del proyecto. Declara las coordenadas del proyecto, la versión de Java y la dependencia del driver JDBC de MySQL.

Cómo defenderlo / qué decir: Maven descarga solo el driver (**mysql-connector-j**) y lo agrega al classpath; sin él, DriverManager no puede hablar con MySQL. Es un driver **tipo 4**: está escrito 100% en Java y habla directo el protocolo de MySQL, sin librerías nativas (portable).

```
1 | <?xml version="1.0" encoding="UTF-8"?>
2 | <project xmlns="http://maven.apache.org/POM/4.0.0"
3 |     xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
4 |     xsi:schemaLocation="http://maven.apache.org/POM/4.0.0 http://maven.apache.org/xsd/maven-4.0.0.xsd">
5 |     <modelVersion>4.0.0</modelVersion>
6 |
```

Líneas 1 a 6: Cabecera XML del POM y versión del modelo (4.0.0).

```
7 |     <groupId>edu.usal</groupId>
8 |     <artifactId>clase-24-06</artifactId>
9 |     <version>1.0-SNAPSHOT</version>
10 |
```

Líneas 7 a 10: Coordenadas del proyecto: grupo **edu.usal**, artefacto **clase-24-06** y versión 1.0-SNAPSHOT (SNAPSHOT = en desarrollo).

```
11 |     <properties>
12 |         <maven.compiler.source>11</maven.compiler.source>
13 |         <maven.compiler.target>11</maven.compiler.target>
14 |         <project.build.sourceEncoding>UTF-8</project.build.sourceEncoding>
15 |     </properties>
16 |
```

Líneas 11 a 16: Propiedades: compila con Java 11 (source y target) y lee los fuentes en UTF-8 (para no romper tildes y eñes).

```
17 |     <dependencies>
18 |         <dependency>
19 |             <groupId>com.mysql</groupId>
20 |             <artifactId>mysql-connector-j</artifactId>
21 |             <version>9.2.0</version>
22 |         </dependency>
23 |     </dependencies>
24 |
```

Líneas 17 a 24: Dependencia del driver **com.mysql:mysql-connector-j** versión 9.2.0. Maven la baja del repositorio central automáticamente.

```
25 | </project>
```

Línea 25: Cierre del POM.

014 - Clase 24-06 / src / main / resources / db.sql

Qué es y para qué sirve: Script SQL que crea la base **universidad**, la tabla **estudiantes** y la llena con 45 estudiantes de prueba.

Cómo defenderlo / qué decir: Sin datos de prueba no se pueden mostrar las consultas. DROP TABLE IF EXISTS permite correr el script varias veces sin error. El id es AUTO_INCREMENT: lo genera la base y no se inserta a mano.

```

1 | CREATE DATABASE IF NOT EXISTS universidad;
2 | USE universidad;
3 |

```

Líneas 1 a 3: Crea la base si no existe y la selecciona con **USE**.

```

4 | DROP TABLE IF EXISTS estudiantes;
5 |

```

Líneas 4 a 5: Borra la tabla si existía, para empezar de cero.

```

6 | CREATE TABLE estudiantes (
7 |             id INT AUTO_INCREMENT PRIMARY KEY,
8 |             nombre VARCHAR(100) NOT NULL,
9 |             edad INT NOT NULL,
10 |            carrera VARCHAR(100) NOT NULL,
11 |            promedio DOUBLE NOT NULL
12 | );
13 |

```

Líneas 6 a 13: Crea la tabla: **id** entero autoincremental y clave primaria; **nombre** y **carrera** de texto (hasta 100), **edad** entera y **promedio** decimal, todos obligatorios (NOT NULL).

```

14 | INSERT INTO estudiantes (nombre, edad, carrera, promedio) VALUES
15 |             ('Lucas Juárez', 21, 'Sistemas', 8.5), ('Anabella
16 |             ↳ Rossi', 23, 'Sistemas', 7.2),
17 |             ('Mateo Fernández', 20, 'Industrial', 6.8), ('Sofía
18 |             ↳ Benítez', 22, 'Mecánica', 9.1),
19 |             ('Diego Gómez', 25, 'Sistemas', 7.9), ('Valentina
20 |             ↳ Luna', 19, 'Química', 8.3),
21 |             ('Bautista Sosa', 24, 'Industrial', 5.4), ('Camila
22 |             ↳ Díaz', 21, 'Sistemas', 9.5),
23 |             ('Thiago Alvarez', 22, 'Mecánica', 6.2), ('Martina
24 |             ↳ Pérez', 20, 'Química', 7.8),
25 |             ('Bruno Torres', 23, 'Sistemas', 8.1), ('Laia Morales',
26 |             ↳ 21, 'Industrial', 8.9),
27 |             ('Joaquín Ríos', 26, 'Mecánica', 6.5), ('Victoria
28 |             ↳ Castro', 20, 'Sistemas', 7.3),
29 |             ('Tomás Ortiz', 22, 'Química', 7.0), ('Mía Silva', 19,
30 |             ↳ 'Industrial', 8.7),
31 |             ('Ian Acosta', 24, 'Sistemas', 5.9), ('Elena Mendoza',
32 |             ↳ 23, 'Mecánica', 9.2),
33 |             ('Agustín Romero', 21, 'Química', 6.4), ('Clara Novoa',
34 |             ↳ 22, 'Sistemas', 8.0),
35 |             ('Facundo Vera', 25, 'Industrial', 7.1), ('Zoe
36 |             ↳ Giménez', 20, 'Mecánica', 8.4),
37 |             ('Leonel Duarte', 21, 'Sistemas', 6.9), ('Emma Burgos',
38 |             ↳ 19, 'Química', 9.0),
39 |             ('Juan Maidana', 27, 'Industrial', 5.2), ('Olivia
40 |             ↳ Peralta', 22, 'Sistemas', 7.6),
41 |             ('Gaspar Medina', 23, 'Mecánica', 7.4), ('Siena
42 |             ↳ Gallardo', 20, 'Química', 8.8),
43 |             ('Mariano Vega', 24, 'Sistemas', 6.7), ('Julia Molina',
44 |             ↳ 21, 'Industrial', 9.3),
45 |             ('Santino Herrera', 22, 'Mecánica', 5.8), ('Almira
46 |             ↳ Flores', 20, 'Química', 7.5),
47 |             ('Ignacio Godoy', 25, 'Sistemas', 8.2), ('Renata
48 |             ↳ Campos', 19, 'Industrial', 8.6),
49 |             ('Francisco Ledesma', 23, 'Mecánica', 6.1), ('Catalina
50 |             ↳ Espinoza', 21, 'Sistemas', 7.7),
51 |             ('Manuel Cabrera', 24, 'Química', 7.2), ('Violeta
52 |             ↳ Prato', 20, 'Industrial', 9.0),
53 |             ('Pedro Núñez', 22, 'Sistemas', 6.3), ('Amparo Ramos',
54 |             ↳ 22, 'Mecánica', 8.1),
55 |             ('Felipe Delgado', 26, 'Química', 5.5), ('Lola Ibáñez',
56 |             ↳ 19, 'Sistemas', 9.4),
57 |             ('Benjamín Gatti', 23, 'Industrial', 7.0), ('Juana
58 |             ↳ Ojeda', 21, 'Mecánica', 7.9),
59 |             ('Marcos Soria', 24, 'Química', 6.6), ('Guadalupe
60 |             ↳ Vidal', 20, 'Sistemas', 8.3),
61 |             ('Lorenzo Ferrari', 22, 'Industrial', 6.0), ('Sara
62 |             ↳ Montenegro', 25, 'Mecánica', 8.8),
63 |             ('Alejo Rossi', 21, 'Química', 7.4), ('Martina Paz',
64 |             ↳ 19, 'Sistemas', 9.7);

```

Líneas 14 a 39: Un único INSERT con muchas filas (una tupla por estudiante): nombre, edad, carrera y promedio. Sirve para tener datos sobre los que consultar y actualizar.

014 - Clase 24-06 / src / main / java / edu / usal / basedatos / connection / MySqlConnectionor.java

Qué es y para qué sirve: Clase que arma y devuelve una conexión (Connection) a la base MySQL universidad.

Cómo defenderlo / qué decir: Centraliza en un único lugar los datos de conexión, para que el resto de las clases solo llamen a getConnection(). Es el antecedente del patrón Singleton / DAO que se ve en la clase siguiente. Mejoras que se pueden mencionar: no dejar la contraseña escrita en el código (usar un archivo .properties o variables de entorno), no devolver null ante un error (lanzar una excepción propia) y usar un pool de conexiones.

```
1 | package edu.usal.basedatos.connection;
2 |
```

Líneas 1 a 2: Declaración del paquete (connection): agrupa lo relativo a la conexión.

```
3 | import java.sql.Connection;
4 | import java.sql.DatabaseMetaData;
5 | import java.sql.DriverManager;
6 | import java.sql.SQLException;
7 |
```

Líneas 3 a 7: Importaciones: **Connection**, **DatabaseMetaData**, **DriverManager**, **SQLException**. Cada import permite usar esa clase por su nombre corto en lugar de escribir el paquete completo.

```
8 | public class MySqlConector {
9 |
```

Líneas 8 a 9: Declara la clase conectora.

```
10 |     public static Connection getConnection(){
11 |         try {
```

Líneas 10 a 11: Método **estático**: se llama sin crear objetos (MySqlConector.getConnection()). Devuelve un objeto **Connection**, la sesión abierta con la base.

```
12 |             Class.forName("com.mysql.cj.jdbc.Driver");
13 |         } catch (ClassNotFoundException e) {
14 |             System.out.println(e.getMessage());
15 |         }
16 |
```

Líneas 12 a 16: Carga en memoria la clase del driver de MySQL (com.mysql.cj.jdbc.Driver). Con los drivers modernos es opcional (se autorregistran), pero es lo que se enseña como paso 1 de JDBC. Si el .jar no está, lanza **ClassNotFoundException**, que se captura y se informa.

```
17 |         String urlJdbc = "jdbc:mysql://localhost:3306/universidad";
18 |         String user = "root";
19 |         String password = "123456";
20 |         Connection connection = null;
21 |
22 |         try {
```

Líneas 17 a 22: Datos de conexión. La URL JDBC tiene el formato **jdbc:mysql://servidor:puerto/base**: acá servidor local, puerto 3306 (el de MySQL) y base universidad. Luego usuario y contraseña, y la variable connection inicializada en null.

```
23 |             connection = DriverManager.getConnection(urlJdbc,
24 |                 user,
25 |                 password);
26 |
27 |
```

Líneas 23 a 27: Paso 2 de JDBC: **DriverManager** busca el driver adecuado según la URL y abre la conexión con usuario y contraseña. Si falla (base apagada, clave incorrecta) lanza **SQLException**.

```
28 |         } catch (SQLException e) {
29 |             System.out.println(e.getMessage());
30 |         }
31 |
```

Líneas 28 a 31: Captura el error y muestra su mensaje por consola.

```

32 |         return connection;
33 |     }
34 |
35 | }

```

Líneas 32 a 35: Devuelve la conexión (o null si falló, algo que conviene evitar). Quien la recibe es responsable de cerrarla con close().

014 - Clase 24-06 / src / main / java / edu / usal / basedatos / statement / StatementQuery.java

Qué es y para qué sirve: Muestra cómo usar **Statement**: un alta (INSERT), una búsqueda (SELECT) y una simulación de inyección SQL.

Cómo defenderlo / qué decir: Sirve para mostrar **por qué no hay que usar Statement con datos del usuario**: el SQL se arma concatenando textos y es vulnerable. También ilustra el ciclo completo: conexión → sentencia → ejecución → resultado → cierre. Para el parcial: **Statement** se reserva para consultas fijas sin parámetros (por ejemplo listar todo).

```

1 | package edu.usal.basedatos.statement;
2 |

```

Líneas 1 a 2: Paquete statement.

```

3 | import edu.usal.basedatos.connection.MySqlConector;
4 |
5 | import java.sql.Connection;
6 | import java.sql.ResultSet;
7 | import java.sql.SQLException;
8 | import java.sql.Statement;
9 |

```

Líneas 3 a 9: Importaciones: **MySqlConector**, **Connection**, **ResultSet**, **SQLException**, **Statement**. Cada import permite usar esa clase por su nombre corto en lugar de escribir el paquete completo.

```

10 | public class StatementQuery {
11 |

```

Líneas 10 a 11: Clase con tres métodos de ejemplo.

```

12 |     public void addEstudiante(String nombre,
13 |                               int edad,
14 |                               String carrera,
15 |                               double promedio) throws SQLException {
16 |

```

Líneas 12 a 16: Alta de un estudiante. Recibe los datos como parámetros del método y declara **throws SQLException** (no la captura: la propaga a quien llama).

```

17 |         String strSQL = "INSERT INTO estudiantes (nombre, edad, carrera, promedio)\n" +
18 |             "VALUES ('" + nombre + "', " + edad + ", '" + carrera + "', " + promedio + ")";
19 |

```

Líneas 17 a 19: Arma el INSERT **concatenando** los valores dentro del texto SQL. Los textos van entre comillas simples ('' + nombre + '') y los números sin comillas. Es la forma insegura: si nombre contiene una comilla o código SQL, rompe o altera la sentencia.

```

20 |         Connection connection = MySqlConector.getConnection();
21 |         Statement statement = connection.createStatement();
22 |

```

Líneas 20 a 22: Pide una conexión y crea el objeto **Statement** con createStatement().

```

23 |         int filasAfectadas = statement.executeUpdate(strSQL);
24 |
25 |         if(filasAfectadas >= 1) {
26 |             System.out.println("Estudiante dado de alta");
27 |         }
28 |

```

Líneas 23 a 28: **executeUpdate(sql)** envía el INSERT y devuelve cuántas filas se afectaron. Si fue 1 o más, informa el alta.

```

29 |         statement.close();
30 |         connection.close();
31 |     }
32 |

```

Líneas 29 a 32: Cierra primero el Statement y después la Connection (ese orden es obligatorio).

```

33 |     public void buscarPorNombre(String nombre) throws SQLException {
34 |         String strSql = "SELECT * FROM estudiantes WHERE nombre = '" + nombre + "'";
35 |
36 |         Connection connection = MySqlConnection.getConnection();
37 |         Statement statement = connection.createStatement();
38 |

```

Líneas 33 a 38: Búsqueda por nombre. Arma el SELECT concatenando el nombre entre comillas simples.

```

39 |         ResultSet resultSet = statement.executeQuery(strSql);
40 |

```

Líneas 39 a 40: `executeQuery(sql)` ejecuta el SELECT y devuelve un **ResultSet**.

```

41 |         while(resultSet.next()){
42 |             int id = resultSet.getInt(1);
43 |             String nombreBd = resultSet.getString("nombre");
44 |             String carrera = resultSet.getString("carrera");
45 |             double promedio = resultSet.getDouble("promedio");
46 |
47 |             System.out.println("Datos del estudiante : " +
48 |                 "ID : " + id + " Nombre : " + nombre + " Carrera : " + carrera
49 |                 + " promedio : " + promedio);
50 |         }
51 |

```

Líneas 41 a 51: Recorre el resultado: `next()` avanza el cursor y devuelve true mientras haya filas. Lee columnas por posición (`getInt(1)`, la primera columna es 1) o por nombre (`getString("nombre")`, `getDouble()`) y las imprime.

```

52 |         statement.close();
53 |         connection.close();
54 |     }
55 |

```

Líneas 52 a 55: Cierre de recursos.

```

56 |     public void sqlI(String user, String pass){
57 |         String strSql = "SELECT * FROM usuarios WHERE username = '"
58 |             + user + "' AND password = '" + pass + "'";
59 |
60 |         System.out.println(strSql);
61 |     }
62 | }

```

Líneas 56 a 62: Demostración de **inyección SQL**: arma el texto de un login concatenando usuario y contraseña y lo imprime (no lo ejecuta). Con la contraseña ' OR '1' = '1' el SQL resultante es ... AND password = " OR '1' = '1', que siempre es verdadero.

014 - Clase 24-06 / src / main / java / edu / usal / basedatos / statement / PreparedStatementQuery.java

Qué es y para qué sirve: Muestra cómo usar **PreparedStatement** con parámetros ? para actualizar el promedio de un estudiante.

Cómo defenderlo / qué decir: Es la forma correcta cuando hay datos del usuario: el SQL está escrito una sola vez con comodines ? y los valores se cargan aparte con `setXxx`. No hay concatenación, así que **no hay inyección SQL**, se evitan errores de comillas y la base puede reutilizar la sentencia precompilada.

```

1 | package edu.usal.basedatos.statement;
2 |

```

Líneas 1 a 2: Paquete statement.


```

3 | import edu.usal.basedatos.connection.MySqlConector;
4 |
5 | import java.sql.Connection;
6 | import java.sql.PreparedStatement;
7 | import java.sql.SQLException;
8 |

```

Líneas 3 a 8: Importaciones: **MySqlConector**, **Connection**, **PreparedStatement**, **SQLException**. Cada import permite usar esa clase por su nombre corto en lugar de escribir el paquete completo.

```

9 | public class PreparedStatementQuery {
10 |

```

Líneas 9 a 10: Clase con un método de ejemplo.

```

11 |     public void actualizar(double promedio, String nombre) throws SQLException {

```

Línea 11: Recibe el nuevo promedio y el nombre del estudiante.

```

12 |         String strSQL = "UPDATE estudiantes SET promedio = ? WHERE nombre = ?";
13 |         Connection connection = MySqlConector.getConnection();

```

Líneas 12 a 13: El SQL con dos comodines **?**: no llevan comillas aunque sean texto; JDBC se encarga.

```

14 |         PreparedStatement preparedStatement = connection.prepareStatement(strSQL);
15 |

```

Líneas 14 a 15: **prepareStatement(sql)** recibe el SQL ya al crear el objeto (a diferencia de Statement). Aquí se precompila.

```

16 |         preparedStatement.setDouble(1, promedio);
17 |         preparedStatement.setString(2, nombre);
18 |

```

Líneas 16 a 18: Asigna los valores: **setXxx(posición, valor)**. La posición del **?** empieza en 1: el 1 es el promedio y el 2 el nombre; el tipo del método (**setDouble**, **setString**) debe coincidir con el de la columna.

```

19 |         int filasAfectadas = preparedStatement.executeUpdate();
20 |
21 |         if(filasAfectadas >= 1){
22 |             System.out.println("Estudiante actualizado con éxito");
23 |         }
24 |

```

Líneas 19 a 24: **executeUpdate()** va **sin argumentos** porque el SQL ya está en el objeto. Devuelve las filas modificadas; si es ≥ 1 informa el éxito.

```

25 |         preparedStatement.close();
26 |         connection.close();
27 |     }
28 | }

```

Líneas 25 a 28: Cierra el PreparedStatement y la Connection.

014 - Clase 24-06 / src / main / java / edu / usal / basedatos / Pruebas.java

Qué es y para qué sirve: Clase con el main que prueba las clases anteriores.

Cómo defenderlo / qué decir: Es el punto de entrada de la demostración. Las líneas comentadas son pruebas que se activan de a una.

```

1 | package edu.usal.basedatos;
2 |

```

Líneas 1 a 2: Paquete base.

```

3 | import edu.usal.basedatos.statement.PreparedStatementQuery;
4 | import edu.usal.basedatos.statement.StatementQuery;
5 |
6 | import java.sql.PreparedStatement;
7 | import java.sql.SQLException;
8 |

```

Líneas 3 a 8: Importaciones: **PreparedStatementQuery**, **StatementQuery**, **PreparedStatement**, **SQLException**. Cada import permite usar esa clase por su nombre corto en lugar de escribir el paquete completo.

```

9 | public class Pruebas {
10 |

```

Líneas 9 a 10: Clase de prueba.

```

11 |     public static void main(String[] args) throws SQLException {
12 |         StatementQuery statementQuery = new StatementQuery();
13 |         //statementQuery.addEstudiante("Estudiante nuevo", 22, "Arquitectura", 6.8);
14 |         //statementQuery.buscarPorNombre("Estudiante nuevo");
15 |

```

Líneas 11 a 15: Punto de entrada. Declara throws SQLException porque los métodos probados la propagan. Crea un StatementQuery y deja comentadas las pruebas de alta y búsqueda.

```

16 |         statementQuery.sqlI("pepito", "OR '1' = '1'");
17 |

```

Líneas 16 a 17: Ejecuta la demostración de inyección SQL con la contraseña maliciosa.

```

18 |         PreparedStatementQuery psQuery = new PreparedStatementQuery();
19 |         //psQuery.actualizar(8.9, "Estudiante nuevo");
20 |     }
21 | }

```

Líneas 18 a 21: Crea el objeto de PreparedStatement (su prueba queda comentada).

Clase 015 - 29-07: CallableStatement y procedimientos almacenados

Se crea un **procedimiento almacenado** en MySQL (sp_procesar_beca) que decide si a un estudiante se le otorga una beca según su promedio, y se lo llama desde Java con **CallableStatement**. El procedimiento usa dos parámetros de entrada (IN) y uno de salida (OUT).

015 - Clase 29-07 / pom.xml

Qué es y para qué sirve: Archivo de configuración de Maven (POM) del proyecto. Declara las coordenadas del proyecto, la versión de Java y la dependencia del driver JDBC de MySQL.

Cómo defenderlo / qué decir: Maven descarga solo el driver (**mysql-connector-j**) y lo agrega al classpath; sin él, DriverManager no puede hablar con MySQL. Es un driver **tipo 4**: está escrito 100% en Java y habla directo el protocolo de MySQL, sin librerías nativas (portable).

```

1 | <?xml version="1.0" encoding="UTF-8"?>
2 | <project xmlns="http://maven.apache.org/POM/4.0.0"
3 |         xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
4 |         xsi:schemaLocation="http://maven.apache.org/POM/4.0.0 http://maven.apache.org/xsd/maven-4.0.0.xsd">
5 |     <modelVersion>4.0.0</modelVersion>
6 |

```

Líneas 1 a 6: Cabecera XML del POM y versión del modelo (4.0.0).

```

7 |     <groupId>edu.usal</groupId>
8 |     <artifactId>clase-29-07</artifactId>
9 |     <version>1.0-SNAPSHOT</version>
10 |

```

Líneas 7 a 10: Coordenadas del proyecto: grupo **edu.usal**, artefacto **clase-29-07** y versión 1.0-SNAPSHOT (SNAPSHOT = en desarrollo).

```

11 | <properties>
12 |   <maven.compiler.source>11</maven.compiler.source>
13 |   <maven.compiler.target>11</maven.compiler.target>
14 |   <project.build.sourceEncoding>UTF-8</project.build.sourceEncoding>
15 | </properties>
16 |

```

Líneas 11 a 16: Propiedades: compila con Java 11 (source y target) y lee los fuentes en UTF-8 (para no romper tildes y eñes).

```

17 | <dependencies>
18 |   <dependency>
19 |     <groupId>com.mysql</groupId>
20 |     <artifactId>mysql-connector-j</artifactId>
21 |     <version>9.2.0</version>
22 |   </dependency>
23 | </dependencies>
24 |

```

Líneas 17 a 24: Dependencia del driver **com.mysql:mysql-connector-j** versión 9.2.0. Maven la baja del repositorio central automáticamente.

```

25 | </project>

```

Línea 25: Cierre del POM.

015 - Clase 29-07 / src / main / resources / sp.sql

Qué es y para qué sirve: Define el procedimiento almacenado **sp_procesar_beca** en la base universidad.

Cómo defenderlo / qué decir: La **regla de negocio** (promedio ≥ 8 para tener beca) vive **dentro de la base**, no en Java. Eso es lo que justifica usar un SP: lógica centralizada y rápida, reutilizable por cualquier aplicación. Observación para la defensa: el script quedó con un DELIMITER ; final sin su DELIMITER \$\$ inicial; en el cliente de MySQL (Workbench/consola) hay que cambiar el delimitador antes de crear el procedimiento para que los ; internos no corten la sentencia.

```

1 | CREATE PROCEDURE sp_procesar_beca(
2 |   IN p_estudiante_id INT,
3 |   IN p_monto_solicitado DECIMAL(10,2),
4 |   OUT p_resultado VARCHAR(100)
5 | )

```

Líneas 1 a 5: Encabezado del procedimiento: se llama **sp_procesar_beca** y tiene tres parámetros: **IN p_estudiante_id** (entero, qué estudiante), **IN p_monto_solicitado** (decimal, cuánto se pide) y **OUT p_resultado** (texto de hasta 100 caracteres con la respuesta, que el SP devuelve a quien lo llama).

```

6 | BEGIN
7 |   DECLARE v_promedio DOUBLE;
8 |

```

Líneas 6 a 8: Inicio del cuerpo. Declara la variable local **v_promedio** (DOUBLE).

```

9 |   -- Obtener el promedio del estudiante
10 |   SELECT promedio INTO v_promedio
11 |   FROM estudiantes
12 |   WHERE id = p_estudiante_id;
13 |

```

Líneas 9 a 13: Consulta con **SELECT ... INTO**: guarda el promedio del estudiante en la variable. Si no existe el estudiante, la variable queda en NULL.

```

14 |   -- Lógica de negocio
15 |   IF v_promedio IS NULL THEN
16 |     SET p_resultado = 'ERROR: El estudiante no existe.';
17 |   ELSEIF v_promedio >= 8.0 THEN
18 |     -- Si califica, podríamos insertar en una tabla de becas o actualizar datos
19 |     SET p_resultado = CONCAT('APROBADO: Beca otorgada por $', p_monto_solicitado);
20 |   ELSE
21 |     SET p_resultado = CONCAT('RECHAZADO: Promedio insuficiente (', v_promedio, '). Requiere >= 8.0');

```

Líneas 14 a 21: Lógica con IF/ELSEIF/ELSE: si el promedio es NULL el estudiante no existe (resultado de error); si es ≥ 8.0 se aprueba la beca y el mensaje se arma con **CONCAT** incluyendo el monto; en otro caso se rechaza informando el promedio y el mínimo requerido. **SET p_resultado = ...** asigna el parámetro de salida.

```
22 | END IF;
23 | END
24 |
```

Líneas 22 a 24: Cierra el procedimiento.

```
25 | DELIMITER ;
```

Línea 25: Restaura el delimitador normal (;). Ver la observación de la defensa.

015 - Clase 29-07 / src / main / java / edu / usal / basedatos / connection / MySqlConnection.java

Qué es y para qué sirve: Clase que arma y devuelve una conexión (Connection) a la base MySQL universidad.

Cómo defenderlo / qué decir: Centraliza en un único lugar los datos de conexión, para que el resto de las clases solo llamen a getConnection(). Es el antecedente del patrón Singleton / DAO que se ve en la clase siguiente. Mejoras que se pueden mencionar: no dejar la contraseña escrita en el código (usar un archivo .properties o variables de entorno), no devolver null ante un error (lanzar una excepción propia) y usar un pool de conexiones.

```
1 | package edu.usal.basedatos.connection;
2 |
```

Líneas 1 a 2: Declaración del paquete (connection): agrupa lo relativo a la conexión.

```
3 | import java.sql.Connection;
4 | import java.sql.DatabaseMetaData;
5 | import java.sql.DriverManager;
6 | import java.sql.SQLException;
7 |
```

Líneas 3 a 7: Importaciones: **Connection**, **DatabaseMetaData**, **DriverManager**, **SQLException**. Cada import permite usar esa clase por su nombre corto en lugar de escribir el paquete completo.

```
8 | public class MySqlConnection {
9 |
```

Líneas 8 a 9: Declara la clase conectora.

```
10 |     public static Connection getConnection(){
11 |         try {
```

Líneas 10 a 11: Método **estático**: se llama sin crear objetos (MySqlConnection.getConnection()). Devuelve un objeto **Connection**, la sesión abierta con la base.

```
12 |             Class.forName("com.mysql.cj.jdbc.Driver");
13 |         } catch (ClassNotFoundException e) {
14 |             System.out.println(e.getMessage());
15 |         }
16 |
```

Líneas 12 a 16: Carga en memoria la clase del driver de MySQL (com.mysql.cj.jdbc.Driver). Con los drivers modernos es opcional (se autorregistran), pero es lo que se enseña como paso 1 de JDBC. Si el .jar no está, lanza **ClassNotFoundException**, que se captura y se informa.

```
17 |         String urlJdbc = "jdbc:mysql://localhost:3306/universidad";
18 |         String user = "root";
19 |         String password = "123456";
20 |         Connection connection = null;
21 |
22 |         try {
```

Líneas 17 a 22: Datos de conexión. La URL JDBC tiene el formato **jdbc:mysql://servidor:puerto/base**: acá servidor local, puerto 3306 (el de MySQL) y base universidad. Luego usuario y contraseña, y la variable connection inicializada en null.

```

23 |         connection = DriverManager.getConnection(urlJdbc,
24 |             user,
25 |             password);
26 |
27 |

```

Líneas 23 a 27: Paso 2 de JDBC: **DriverManager** busca el driver adecuado según la URL y abre la conexión con usuario y contraseña. Si falla (base apagada, clave incorrecta) lanza **SQLException**.

```

28 |         } catch (SQLException e) {
29 |             System.out.println(e.getMessage());
30 |         }
31 |

```

Líneas 28 a 31: Captura el error y muestra su mensaje por consola.

```

32 |         return connection;
33 |     }
34 |
35 | }

```

Líneas 32 a 35: Devuelve la conexión (o null si falló, algo que conviene evitar). Quien la recibe es responsable de cerrarla con close().

015 - Clase 29-07 / src / main / java / edu / usal / basedatos / callable / CallableSt.java

Qué es y para qué sirve: Llama al procedimiento almacenado sp_procesar_beca usando **CallableStatement** y devuelve el texto de resultado.

Cómo defenderlo / qué decir: Es el modelo a seguir para el punto «uso de CallableStatement» del parcial. Pasos que hay que saber de memoria: **(1)** prepareCall con la sintaxis { call sp(?, ?, ?) }; **(2)** setXxx para los IN; **(3)** registerOutParameter para los OUT **antes** de ejecutar; **(4)** execute(); **(5)** getXxx para leer los OUT; **(6)** cerrar.

```

1 | package edu.usal.basedatos.callable;
2 |

```

Líneas 1 a 2: Paquete callable.

```

3 | import edu.usal.basedatos.connection.MySqlConector;
4 |
5 | import java.sql.CallableStatement;
6 | import java.sql.Connection;
7 | import java.sql.SQLException;
8 | import java.sql.Types;
9 |

```

Líneas 3 a 9: Importaciones: **MySqlConector**, **CallableStatement**, **Connection**, **SQLException**, **Types**. Cada import permite usar esa clase por su nombre corto en lugar de escribir el paquete completo.

```

10 | public class CallableSt {
11 |

```

Líneas 10 a 11: Clase que encapsula la llamada al SP.

```

12 |     public String verificarBeca(int id, double montoSolicitado) throws SQLException {
13 |         Connection connection = MySqlConector.getConnection();
14 |

```

Líneas 12 a 14: Recibe el id del estudiante y el monto y devuelve el mensaje de resultado. Propaga **SQLException**.

```

15 |         String strSP = "{ call sp_procesar_beca(?, ?, ?)}";
16 |
17 |         //Instanciamos el callable

```

Líneas 15 a 17: Sintaxis estándar JDBC para llamar a un SP: { call nombre(?, ?, ?) }, un ? por parámetro (los dos IN y el OUT).

```

18 | CallableStatement callableStatement = connection.prepareCall(strSP);
19 |
20 | //Seteo de los comodines

```

Líneas 18 a 20: `prepareCall` crea el `CallableStatement` a partir del texto del SP.

```

21 | callableStatement.setInt(1, id);
22 | callableStatement.setDouble(2, montoSolicitado);
23 |
24 | //3 PARAM del SP - OUT

```

Líneas 21 a 24: Carga los parámetros de **entrada**: posición 1 = id, posición 2 = monto.

```

25 | callableStatement.registerOutParameter(3, Types.VARCHAR);
26 |
27 | //Exec del SP

```

Líneas 25 a 27: Registra el tercer parámetro como **de salida** de tipo VARCHAR (`java.sql.Types`). Si no se registra antes de ejecutar, falla.

```

28 | callableStatement.execute();
29 |
30 | //Obtener resultado del SP

```

Líneas 28 a 30: Ejecuta el procedimiento en la base. Se usa `execute()`.

```

31 | String resultado = callableStatement.getString(3);
32 |

```

Líneas 31 a 32: Lee el parámetro de salida con `getString(3)` (la posición del OUT).

```

33 | callableStatement.close();
34 | connection.close();
35 |

```

Líneas 33 a 35: Cierra el `CallableStatement` y la conexión.

```

36 |         return resultado;
37 |     }
38 | }

```

Líneas 36 a 38: Devuelve el mensaje al que llamó.

015 - Clase 29-07 / src / main / java / edu / usal / basedatos / PruebaSP.java

Qué es y para qué sirve: main que prueba el procedimiento y deja, como comentario, un ejercicio de práctica sobre qué tipo de sentencia usar en cada caso.

Cómo defenderlo / qué decir: El comentario es oro para el parcial: enseña a **elegir la sentencia correcta**. Listar todo (SELECT sin parámetros) → Statement o PreparedStatement con `executeQuery`. Buscar por parte del título (LIKE con parámetros) → PreparedStatement con `executeQuery`. Registrar un préstamo con un SP y obtener la respuesta → CallableStatement con `execute`.

```

1 | package edu.usal.basedatos;
2 |

```

Líneas 1 a 2: Paquete base.

```

3 | import edu.usal.basedatos.callable.CallableSt;
4 |
5 | import java.sql.SQLException;
6 |

```

Líneas 3 a 6: Importaciones: **CallableSt**, **SQLException**. Cada import permite usar esa clase por su nombre corto en lugar de escribir el paquete completo.

```

7 | public class PruebaSP {
8 |

```

Líneas 7 a 8: Clase de prueba.

```

9 | public static void main(String[] args) throws SQLException {
10 |     CallableSt callableSt = new CallableSt();
11 |     String resultado = callableSt.verificarBeca(47, 5000.0);
12 |     System.out.println("RESULTADO SP : " + resultado);
13 | }
14 |

```

Líneas 9 a 14: Crea el objeto, llama a verificarBeca(47, 5000.0) (estudiante 47, monto \$5000) e imprime la respuesta del SP.

```

15 | /*
16 |
17 |     PUNTO 1: Obtener un listado general de todos los libros registrados con su stock.
18 |     - SELECT * FROM libros;
19 |     - Resolución : PS, executeQuery() [] || ST, executeQuery() [X]
20 |
21 |     PUNTO 2: Permitir al usuario buscar libros introduciendo parte del titulo o del autor.
22 |     - SELECT * FROM libros WHERE titulo LIKE '%titulo%' OR autor LIKE '%autor%'
23 |     - Resolución : PS, executeQuery()
24 |
25 |     PUNTO 3: Ejecutar el SP para registrar el prestamo de un libro a un usuario y obtener
26 |     el resultado de respuesta.
27 |     - Resolución : Callable, execute()
28 |
29 | */
30 | }

```

Líneas 15 a 30: Comentario de bloque con el ejercicio: para cada punto se indica la sentencia y el método de ejecución correctos. Punto 1: listado general, no lleva parámetros. Punto 2: búsqueda con LIKE y parámetros, por lo que PreparedStatement. Punto 3: ejecutar un SP, por lo que CallableStatement con execute().

Resumen para estudiar

- JDBC = API estándar (java.sql) + un driver por motor. Tipo 4 = todo Java.
- Pasos: driver → Connection → sentencia → ejecutar → ResultSet → cerrar (sentencia primero, conexión después).
- **Statement**: SQL fijo; vulnerable si se concatena. **PreparedStatement**: parámetros ?, seguro y rápido. **CallableStatement**: SP con IN/OUT.
- executeQuery → ResultSet; executeUpdate → filas afectadas; execute → boolean.
- ResultSet: next(), getXxx desde la posición 1; ResultSetMetaData para describir columnas.

Mejoras del código de clase (para sumar puntos en la defensa)

- Cerrar recursos con **try-with-resources** para que se cierren aunque ocurra una excepción (en las clases de ejemplo, si falla el SQL, la conexión queda abierta).
- No hardcodear usuario y contraseña: usar un archivo .properties o variables de entorno.
- No devolver null ante un error de conexión: lanzar una excepción propia.
- En buscarPorNombre el mensaje imprime el parámetro nombre en lugar del nombreBd leído de la base (inofensivo acá porque son iguales, pero es una imprecisión).

Preguntas típicas

¿Cuál es la diferencia entre Statement y PreparedStatement?

Statement se arma concatenando y se compila en cada ejecución; es vulnerable a inyección SQL. PreparedStatement usa parámetros ?, se precompila, es más rápido al repetirse y es seguro.

¿Por qué se cierra la Statement antes que la Connection?

Porque la sentencia depende de la conexión: cerrar la conexión primero deja la sentencia inválida y provoca una excepción.

¿Para qué sirve registerOutParameter?

Para avisarle a JDBC que un parámetro del SP es de salida y de qué tipo, antes de ejecutar; luego se lee con getXxx.